Development of an Edge Computing-Based Voice and Gesture-Controlled Lighting System

Anh Q. Vu*, Hung D. Truong*, Hoang H.N. Le*, Le The Dung†
*FPT University, Ho Chi Minh City, Vietnam
†HCMC University of Technology and Education, Ho Chi Minh City, Vietnam
Corresponding author: Le The Dung, Email: dunglt@hcmute.edu.vn

***Abstract*—This paper presents a contactless smart lighting system controlled by voice and hand gestures. The architecture integrates an ESP32, a microphone, an optical gesture sensor, a relay stage for a 12V–10W LED load, and a web monitoring interface. The on-device keyword-spotting model is a quantized Int8 1D-CNN over MFCCs, compiled by the Edge Impulse EON™ Compiler into hardware-aware C++. The work also contributes a cost model that attributes the deployed network's latency and memory, and a command arbitration policy for concurrent commands. The proposed system achieves an overall classification accuracy of 88.2% with an inference latency of 7 ms, a fourfold reduction of the inference stage relative to a contemporary TinyML implementation. Amdahl's law bounds the complete cycle gain at 7.9%, against a measured 3.4%, because the MFCC front-end dominates the arithmetic. Furthermore, with peak RAM usage of just 3.8 KB and Flash usage of 31.2 KB, the model achieves a competitive level of memory efficiency. Dynamic memory stays stable in continuous operation, confirming real-time multimodal edge AI on the ESP32's dual-core architecture.**

***Index Terms*—Edge Computing, ESP32, 1D-CNN, MFCC, Keyword Spotting, Gesture Recognition, Smart Lighting, WebSocket, IoT.**

I. Introduction

Human-machine interaction in smart environments is moving from mechanical switches toward contactless control. Shared contact surfaces are recognized vectors for pathogen transfer, motivating this shift in hygiene-critical settings. Cloud-based voice control adds latency, needs connectivity, and sends private audio off-device. Tiny machine learning removes these dependencies by running inference locally, where the microcontroller memory hierarchy is the binding constraint [1].

Prior work established the design space for microcontroller keyword spotting [2], extended it through architecture search [3], and standardized evaluation [4]. On the ESP32, González *et al.* [5] deployed an MFCC-based recognizer with post-training quantization. Two deficiencies occur in this literature. First, deployments are predominantly unimodal. Running two asynchronous pipelines on one device introduces memory contention and a control problem, since independent sources may command conflicting states in the same interval. Second, results are reported as headline latency and footprint figures without an accompanying cost model, which prevents attribution of an observed improvement to the network, the quantization, the code generator, or the measurement boundary.

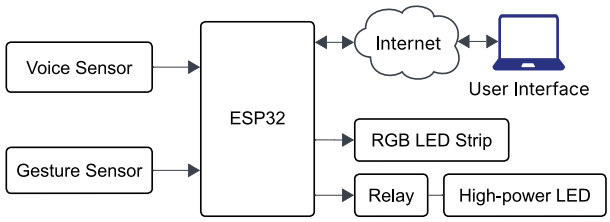


Fig. 1. Overall architecture and data workflow of the proposed multimodal system.

This paper reports a complete multimodal edge node and, in place of a claim of algorithmic novelty, the analysis required to interpret its measurements. The contributions are threefold. First, an integrated node fuses I2S acoustic sampling with interrupt-driven I2C gesture sensing on a single ESP32 and serves a firmware-resident WebSocket interface, with all inference performed locally. Second, a command arbitration policy discussed in Section IV resolves concurrent conflicting commands through single-writer ownership, idempotent commands, windowed priority, and per-actuator lockout. Third, a cost model presented in Section V derives the parameter count, multiply-accumulate count, and tensor liveness limit of the deployed network and compares each against the measurement. It establishes that the feature front end incurs $12.6\times$ the network arithmetic, that 70% of the reported arena and 75% of the flash are not model attributable, and that the arithmetic reduction over the reference topology over-predicts the measured latency by roughly twofold. By Amdahl's law [11], the fourfold inference speedup yields at most a 7.9% end-to-end reduction, against a measured 3.4%.

II. System Architecture

Fig. 1 shows the proposed system, integrating sensing, processing, actuation, and communication. Audio is captured by an INMP441 digital microphone [16] over the I2S bus. Gestures are acquired by a PAJ7620U2 optical sensor [15] via I2C, which asserts an active-low interrupt when a gesture is available. The LED is switched by an isolated relay, and a WS2812B strip provides status feedback.

Inference latency is not interpretable without the execution environment, since clock frequency, flash mode, and core assignment alter it by large factors (Table I). A firmware-resident WebSocket endpoint [14] enables the device to push state for each physical event. The communication state

TABLE I
HARDWARE AND SOFTWARE ENVIRONMENT

Parameter	Value	Parameter	Value
Module	ESP32-WROOM-32E	Framework	ESP-IDF v4.4.x, Arduino core 2.0.x
System on chip	ESP32-D0WD-V3		
Core, clock	Xtensa LX6 dual, 240 MHz, both used	RTOS	FreeRTOS, 1000 Hz tick
		Engine	EON™ Compiler, static C++
SRAM	520 kB internal	Kernels	ESP-NN integer path
PSRAM	none fitted	Compiler	xtensa-esp32-elf-gcc `-O2`
Flash	4 MB, QIO, 80 MHz	Core 0	Wi-Fi stack, WebSocket, arbitration
FPU, SIMD	single precision, no SIMD	Core 1	I2S, MFCC, inference
Load, rails	12 V 10 W at 0.83 A; 12 V, 5 V, 3.3 V	Microphone	INMP441, I2S, 16 kHz mono
		Gesture	PAJ7620U2, I2C 400 kHz, INT

evolves as $s_{k+1} = g(s_k, c_k)$, where s_k is the device state and c_k is the command admitted at step k.

III. PROCESSING PIPELINE

The on-device path from a raw acoustic window to an actuated command has four stages, feature extraction, the network architecture, quantization and code generation via the EON™ Compiler, and the parallel gesture path. All processing is local. The arithmetic and memory figures are derived analytically and cross-checked against the measurements of Section VI.

A. Feature Extraction

The signal is sampled at $f_s = 16\,\mathrm{kHz}$. A pre-emphasis filter $x_{\mathrm{pre}}[n] = x[n] - 0.98\,x[n-1]$ compensates the spectral tilt of voiced speech. Framing at $25\,\mathrm{ms}$ with a $20\,\mathrm{ms}$ hop yields $N = 400$ and $H = 320$ samples, so a $1000\,\mathrm{ms}$ window produces $M = \lfloor (16000 - 400)/320 \rfloor + 1 = 49$ frames. Each frame is transformed by a 512-point fast Fourier transform and projected onto 32 Mel-spaced triangular filters [10] through $m = 2595 \log_{10}(1 + f/700)$, band-limited from $80\,\mathrm{Hz}$ to $4000\,\mathrm{Hz}$ to retain the first three formants and the frication energy that discriminates the plosive onsets. A discrete cosine transform decorrelates the log filterbank energies, and 13 cepstral coefficients are retained, giving $49 \times 13 = 637$ features.

B. Network Architecture

The feature vector is reshaped to $X_{\mathrm{seq}} \in \mathbb{R}^{T \times D}$ with $T = 49$ and $D = 13$, so convolution operates along time with the cepstral index as channel dimension,

$$y_j[n] = \sigma\left(\sum_i \sum_{k=0}^{K-1} w_{j,i}[k]\, x_i[n-k] + b_j \right), \qquad (1)$$

with $\sigma(z) = \max(0, z)$. Two convolutional blocks of 8 and 16 filters with kernel length 3 are each followed by pooling and dropout at $p = 0.25$, terminating in a dense layer over six classes. Each hyperparameter is quantified in Table IV. Thirteen cepstral coefficients are retained because higher indices encode fine spectral detail tied to speaker identity rather than phonetic content, and 13 keeps the first convolution below $16\,\mathrm{k}$ operations. Kernel length 3 spans $60\,\mathrm{ms}$ at a $20\,\mathrm{ms}$ hop, matching a Vietnamese syllable nucleus transition, whereas length 5 spans $100\,\mathrm{ms}$ and crosses syllable boundaries in a bisyllabic command. Two blocks reduce 49 frames to 12, yielding a second layer receptive field near $240\,\mathrm{ms}$. Filter counts of 8 and 16 suit a closed six-class vocabulary, and widening to 16 and 32 increases the operation count by $2.7\times$ (Table IV).

The numerical range is controlled at two points. Each first-convolution output sums only $KC_{\mathrm{in}} = 39$ products of two signed 8-bit values, whose worst-case magnitude is much smaller than the range of the 32-bit accumulator, so overflow cannot occur. The result is then scaled back to 8 bits. The per-tensor scale factor and the saturating clamp of (2) map any out-of-range value to the nearest bound in $[a_{\min}, 127]$ without wrapping around. The ReLU is not a separate stage because $\max(0, z)$ commutes with the affine requantization. It is realized by raising the lower clamp bound to $a_{\min} = Z_{\mathrm{out}}$, discarding negatives without a dedicated tensor pass.

C. Quantization and Code Generation

The model was trained in Edge Impulse Studio using Adam with a learning rate of 0.005 and a batch size of 128. Post-training quantization maps a real tensor to signed 8-bit integers [9] through $r = S(q - Z)$, with $S = (r_{\max} - r_{\min})/255$ and $Z = \mathrm{round}(-128 - r_{\min}/S)$ per tensor. Convolution accumulates into 32-bit registers using ESP-NN kernels compatible with the CMSIS-NN convention [8], and the accumulator is requantized by

$$q_{\mathrm{out}} = \mathrm{clamp}(\mathrm{round}(M q_{\mathrm{acc}}) + Z_{\mathrm{out}},\, a_{\min},\, 127)\,, \qquad (2)$$

where M is a fixed-point multiplier and shift and $a_{\min} = Z_{\mathrm{out}}$ fuses the activation. The quantized graph is compiled by the Edge Impulse EON™ Compiler, which emits C++ invoking the kernels directly rather than traversing a serialized graph at run time as the TFLite Micro interpreter does [7], sizing the arena at compile time and eliminating per-operator dispatch. A command is emitted when the maximum softmax posterior exceeds $\tau = 0.70$.

D. Gesture Path

Alongside the acoustic path, a second modality runs concurrently on the other core and must be acquired without disturbing feature extraction. Since continuous I2C polling would steal cycles from that core, gesture acquisition is interrupt-driven. An active-low edge triggers a service routine that posts an event and returns, while the slower I2C read of the sensor's gesture register is deferred to a task. This limits the interrupt-masked interval. The physical latency is calculated as

$$t_{\mathrm{total}} = t_{\mathrm{acq}} + t_{\mathrm{feat}} + t_{\mathrm{infer}} + t_{\mathrm{arb}} + t_{\mathrm{act}}. \qquad (3)$$

IV. COMMAND ARBITRATION

The pipeline of Section III produces commands from three uncoordinated sources, the classifier on core 1, the gesture interrupt, and a WebSocket frame on core 0. The arbitration

term t_{arb} of (3) is the cost of reconciling them. Three failure modes arise. Events are lost when an interrupt fires during service of a prior event and is not queued. State diverges when the browser applies a command optimistically while the device rejects it. Actuators thrash when a command and its complement arrive within the relay settling time. Four rules eliminate these modes.

Single writer. Each source places an event $e = \langle \mathrm{src}, \mathrm{cmd}, t, \kappa \rangle$, with a microsecond timestamp and confidence, into one shared queue drained by a single arbitration task on core 0, which removes write conflicts between sources. The `FromISR` primitive lets interrupt context enqueue without dropping events, and the queue depth sets the largest burst absorbed without loss.

Idempotent commands. The protocol carries `set(on)`, `set(off)`, and `set(color)` rather than `toggle`, so repeated commands converge to the same state.

Priority within a window. Events within $W = 50\,\mathrm{ms}$ are treated as simultaneous and resolved by the fixed priority $\mathrm{web} \succ \mathrm{gesture} \succ \mathrm{voice}$, which ranks channels by how likely each is to be wrong. A browser command is deliberate and unambiguous, a gesture code derives from a deterministic sensor state machine, and a voice command derives from a classifier with a 12% error rate at the operating point. Equal-priority ties break on the later timestamp, then the higher confidence κ.

Per-actuator lockout. After application, further commands to that actuator are rejected for $L = 300\,\mathrm{ms}$, longer than the relay settling time, so it cannot switch back before settling. Rejected events are logged with their cause, making the loss rate observable.

After each application, the task broadcasts the new device state, and the browser renders only this device-reported state rather than updating optimistically. Therefore, divergence is prevented by construction while the WebSocket link is up. Each arbitration costs one queue operation and a scan over the three sources, giving $t_{\mathrm{arb}} = 54\,\mathrm{ms}$, dominated by the $50\,\mathrm{ms}$ window W.

V. Complexity and Memory Analysis

This section reports the latencies of (3) and the memory of Section II as reproducible figures, derived in closed form from the cost model (4) and cross-checked against on-device measurement. The same model is applied to the reference topology [5].

A. Operation Count

For a one-dimensional convolution with kernel length K, input channels C_{in}, output channels C_{out}, and output positions T_{out}, the number of parameters P and multiply-accumulate operations MAC are

$$P = KC_{\mathrm{in}}C_{\mathrm{out}} + C_{\mathrm{out}}, \quad \mathrm{MAC} = T_{\mathrm{out}}KC_{\mathrm{in}}C_{\mathrm{out}}. \tag{4}$$

Applying (4) per layer as in Table II gives 1878 parameters and 25 656 MAC per inference. At $240\,\mathrm{MHz}$, a measured inference time of $7\,\mathrm{ms}$ for the deployed network implies 65 cycles per MAC, against one to four for an optimized integer kernel.

TABLE II
Layer-by-Layer Cost of the Deployed Network

Layer	Output	Weights	Biases	MAC
MFCC feature tensor	49×13	–	–	–
Conv1D, 8, k=3	49×8	312	8	15 288
MaxPool1D, stride 2	24×8	–	–	–
Conv1D, 16, k=3	24×16	384	16	9 216
MaxPool1D, stride 2	12×16	–	–	–
Dense, 6	6	1 152	6	1 152
Total		**1 848**	**30**	**25 656**

Arithmetic is therefore at most 6% of the interval, the rest overhead. The stage is overhead-bound, not compute-bound, limiting attribution of the speedup to arithmetic alone.

B. Front-End Cost

The network is only part of the arithmetic. Cost figures conventionally omit the feature front end (Fig. 2a). A radix-2 transform of length N requires $\frac{N}{2}\log_2 N$ butterflies of four real multiplications halved by the real input optimization, giving $\approx$ 4608 multiplications per frame. Including pre-emphasis and windowing at 400 operations each, the magnitude spectrum at $N/2$, the filterbank at $\approx 2(N/2+1)$, and the 13×32 cosine transform yield ≈ 6594 operations per frame, or 3.23×10^5 over 49 frames. This is $12.6\times$ the arithmetic of the network, so network reduction yields a bounded return. The front-end parameters, namely transform length, hop, and window, are the effective targets.

C. Memory Accounting

The same accounting applies to memory as to arithmetic, so neither storage figure is model-dominated. Since the generated code double-buffers, the peak is the largest pair of consecutive activations, which are the 637-byte input and the 392-byte first-convolution output. Adding a 156-byte scratch buffer gives an analytic requirement of 1185 bytes against a measured 3891 bytes. The remaining 2706 bytes (70%) are alignment padding, allocator metadata, and quantization tables, none scaling with the model. For flash, weights and biases occupy 1968 bytes and the tables at most 6144 bytes, so $23.3\,\mathrm{kB}$, or 75%, of the $31.2\,\mathrm{kB}$ footprint is generated code and metadata. The headline memory of a compiled model is therefore largely a toolchain property, so a comparison on peak arena is dominated by allocator behavior rather than design.

D. Attribution of the Speedup

With per-stage costs established, the measured speedup can now be attributed. The reference implementation [5] employs four convolutional blocks with filter counts $16 \to 32 \to 64 \to 64$. Evaluating (4) for that topology under identical input dimensions, kernel length, and pooling yields $\approx 21\,900$ parameters and 2.16×10^5 operations, so the deployed network performs $8.4\times$ less arithmetic. The measured latencies are $28\,\mathrm{ms}$ and $7\,\mathrm{ms}$, a reproducible $4.0\times$ reduction of the inference stage. The arithmetic reduction thus over-predicts the measured speedup by $2.1\times$ because fixed per-layer costs dominate a small network (65 cycles per MAC here), and narrow layers use the integer kernels less efficiently than

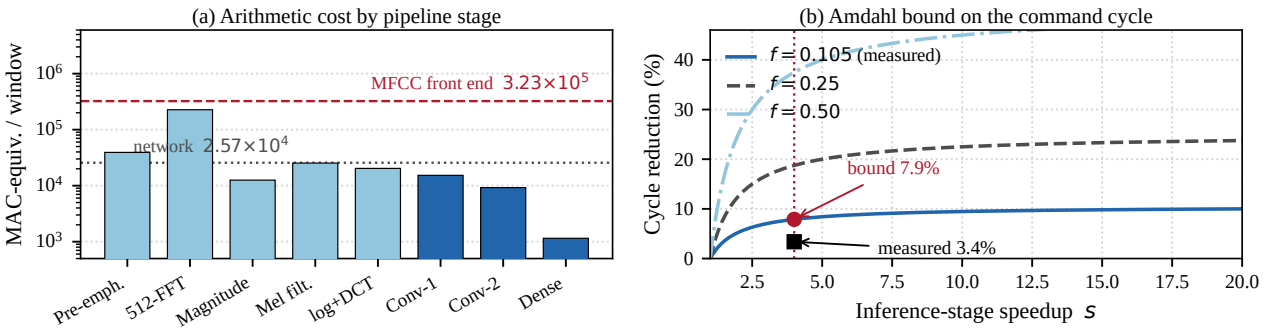


Fig. 2. (a) Arithmetic cost per one-second window by pipeline stage (light, feature extraction; dark, inference), on a log scale. (b) End-to-end cycle reduction against inference-stage speedup, for three values of the inference share f.

wide ones. Because the two systems differ simultaneously in topology, execution engine, kernel library, and toolchain, the present data cannot partition the gain. Therefore, the speedup is attributable neither solely to the code generator nor solely to the architecture.

E. End-to-End Bound

This speedup acts on a single stage of the command cycle, so its effect on total latency is limited. Inference constitutes a fraction $f = 28/266 = 0.105$ of the reference cycle. By Amdahl's law [11], an inference speedup s yields $S(s) = [(1-f)+f/s]^{-1}$, so $s = 4$ bounds the cycle reduction at 7.9%, against a measured reduction from 266 to 257 ms, or 3.4% (Fig. 2b). Consistent with the front-end dominance above, the present system spends 12 ms more outside inference (250 ms vs 238 ms), which cancels most of the 21 ms saved within it, leaving a 9 ms (3.4%) net reduction. A stage-level speedup of this size therefore does not translate into a comparable end-to-end gain. The defensible statements are a fourfold reduction in the inference stage and a 3.4% reduction in the complete cycle, with the reclaimed processor time allowing the gesture path and the WebSocket server to run concurrently with the acoustic pipeline.

VI. Experimental Results

This section validates the analysis of Section V against measurement across four evaluation modes. First, in Subsections VI-A and VI-C, the model was trained on the Edge Impulse platform and deployed to the ESP32, where accuracy, latency, and memory were measured on-device and tabulated beside the derived quantities in Table III. Second, Subsection VI-B evaluates the alternative topologies of Table IV analytically from (4), so only their cost is compared, not their accuracy. Third, Subsection VI-D assesses concurrent command handling with a discrete-event model of the arbitration logic and the scheme it replaces. Finally, Subsection VI-E positions the system against prior work on parameter count.

A. Corpus and Statistical Precision

The corpus reported in Table III holds 720 one-second utterances from four speakers under quiet conditions. The inference latency is the classification time measured on the ESP32 by the Edge Impulse SDK's `run_classifier`. Two

TABLE III
Corpus, Measured Performance, and Derived Quantities

Property	Value
Corpus	
Labels; utterances per label	6; 120 (1 s each)
Total audio; test partition	12 min, 720 utt.; 144 utt.
Speakers; condition	4 (3 M, 1 F); quiet, no added noise
Split	80/20 over recordings, speakers shared
Measured	
Overall accuracy	**88.2%**
Weighted precision / recall / F1	0.89 / 0.88 / 0.88
Area under ROC curve; loss	0.99; 0.39
Inference time (EON, 240 MHz)	**7 ms**
Peak arena; flash	**3.8 kB**; **31.2 kB**
Free heap boot / load / min. block	208.65 / 177.95 / 107.99 kB
Derived (Section V)	
Parameters; MAC per inference	1878; 25 656
MFCC front end MAC per window	3.23×10^5
Analytic arena requirement	1185 B of 3891 B
Cycles per MAC; reference MAC	65; 2.16×10^5

properties qualify the reported accuracy. First, the 80/20 split is over recordings rather than speakers, so the 88.2% reflects within-speaker recall, not generalization to an unenrolled speaker. Second, the 144-item test partition carries substantial sampling error. Treating each trial as Bernoulli, the Wilson interval for $\hat{p} = 0.882$ at $n = 144$ is

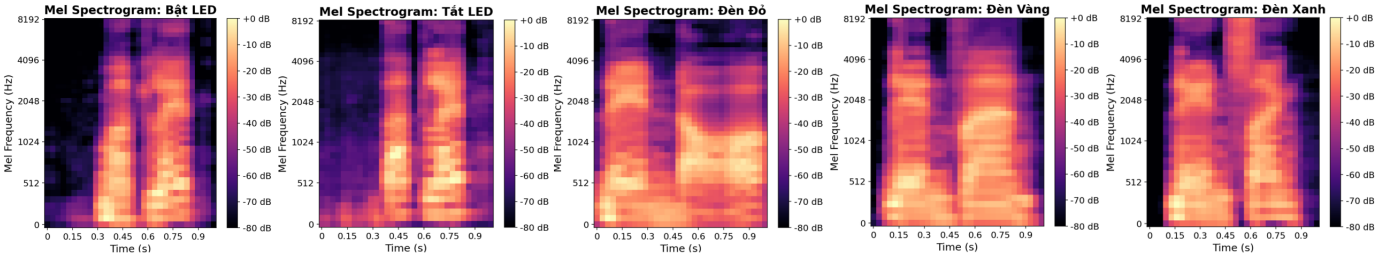
$$\mathrm{CI}_{95} = [\,81.9\%,\ 92.5\%\,], \tag{5}$$

a width of ± 5.3 points. The 1.06 point margin over the 87.14% of [5] lies within this interval, so the difference is not statistically resolved [12]. Each class has $n = 24$ test items, so its confidence interval is wider, 16 to 35 points. Therefore, the ordering in Fig. 4 is indicative rather than statistically firm. Equation (5) excludes variance across training runs, which Section VII identifies as outstanding.

The command set is *bat_led* (on), *tat_led* (off), *den_do* (red), *den_vang* (yellow), *den_xanh* (green), and ambient noise. In Fig. 3, the actuation commands exhibit sharp broadband plosive onsets, whereas the color commands share the prefix *den*, so discrimination resides in the second-syllable vowel formant trajectory.

B. Architecture Cost Analysis

Table IV quantifies alternative configurations via (4). Doubling the filters to $16 \rightarrow 32$ incurs $2.7\times$ the arithmetic, a



(a) bat_led (b) tat_led (c) den_do (d) den_vang (e) den_xanh

Fig. 3. Mel spectrograms of the five voice commands before 1D-CNN processing, labeled (a)–(e).

TABLE IV
ANALYTIC COST OF ALTERNATIVE LIGHTWEIGHT TOPOLOGIES

Configuration	Params	MAC	Arena ≥
8→16, k=3, 13 coeff. (deployed)	**1878**	**25 656**	**1185 B**
16→32, k=3, 13 coeff.	4518	69 744	1577 B
8→16→32, k=3, 13 coeff.	3446	44 088	1185 B
8→16, k=5, 13 coeff.	2342	41 992	1289 B
8→16, k=3, 20 coeff.	2046	33 888	1612 B
16→32→64→64 (reference [5])	21 926	216 048	1577 B

All entries are analytically derived from (4) to compare cost, not accuracy.

TABLE V
SIMULATED CONCURRENT COMMAND ARBITRATION

		Lost		Agree	
Condition	Trials	direct	arb.	direct	arb.
Voice, gesture	2400	0%	**0%**	100%	**100%**
Voice, web	2400	0%	**0%**	50%	**100%**
Gesture, web	2400	0%	**0%**	50%	**100%**
All three	7200	0%	**0%**	65.7%	**100%**
Gesture burst	2400	33.3%	**0%**	100%	**100%**

Agreement is the browser–device match rate, simulation only.

third block 1.7×, kernel length 5 1.6×, and 20 coefficients 1.3×, while enlarging the arena by 36%. The deployed configuration is minimal on every axis. The table quantifies cost but not accuracy, which would require retraining. A richer configuration is therefore justified only if its accuracy gain outweighs the added cost shown here.

C. Training Dynamics and Class Behavior

Over 100 epochs, accuracy plateaus near 0.85 with most of the gain in the first 20 epochs, and cross-entropy loss falls to about 0.40, matching 0.39 in Table III. The training-to-validation gap stays within roughly one accuracy point, indicating that dropout at $p = 0.25$ is adequate regularization at this corpus scale, though the narrow gap does not establish generalization to unseen speakers or conditions. Fig. 4 presents class behavior. The *den_do* class attains the lowest F1 score, 0.80, misclassified as *den_xanh* (16.9%) and *den_vang* (11.9%), all sharing the *den* prefix. The actuation commands attain 0.93 and 0.89, and noise 0.92. Within the color group, *den_xanh* attains 98.6% recall at 0.78 implied precision, an attractor for the confusable mass of the remaining colors.

A small network suffices because the MFCC front end already separates most classes. The confusion structure shows that few parameters are needed once the features are separable, and quantization then costs little accuracy. Heap profiling reports 208.65 kB free after boot, settling to 177.95 kB under sustained load, with a minimum contiguous block of 107.99 kB. The latter is operative because a large free heap does not prevent allocation failure under fragmentation.

D. Simulated Arbitration Verification

After analyzing recognition, this subsection turns to the control logic of Section IV. Whether it admits lost events or state divergence is a property of the algorithm, and can be established without hardware by running the arbitration policy and the direct scheme it replaces as a discrete-event model on identical traces. On-device injection, the definitive confirmation, is left to future work. The model represents the interrupt service routine as a 60 μs critical section (a two-byte I2C read at 400 kHz), actuator service as 120 μs, and the broadcast path as a 4 ms one-way delay. In the direct scheme, every source writes the actuator, the browser updates optimistically on click, and the gesture register is read inside the interrupt, so a second report under masking is lost. The arbitrated scheme applies the four rules of Section IV with queue depth 16, $W = 50$ ms, and $L = 300$ ms. Command pairs were injected at offsets $\Delta t \in \{0, 10, 25, 50, 100, 250\}$ ms across every ordered source pair, 200 trials each, plus a burst of two gesture reports 20 to 2000 μs apart. Both schemes were implemented as a discrete-event model in Python and run on identical command sequences with a fixed random seed.

The direct scheme diverges in half of all browser-involving trials, since the browser commits to its optimistic view while another source alters the device, and in 34.3% of three-source trials. It loses one gesture in three during a burst. The arbitrated scheme incurs no lost events and no divergence, the expected consequence of single-writer ownership and idempotent commands. Its 54 ms median arbitration latency is dominated by the decision window and is only 21% of the command cycle, acceptable here though not for a faster actuator.

E. Comparison with Prior Work

Table VI positions ours against the ESP32 implementation of [5] and reference keyword-spotting models. Parameter counts are comparable across rows, derived through (4), but accuracy is not, because vocabularies, languages, and class counts differ. The deployed model is one to two orders of magnitude smaller, on a custom six-class vocabulary rather than the twelve-class Speech Commands benchmark [6].

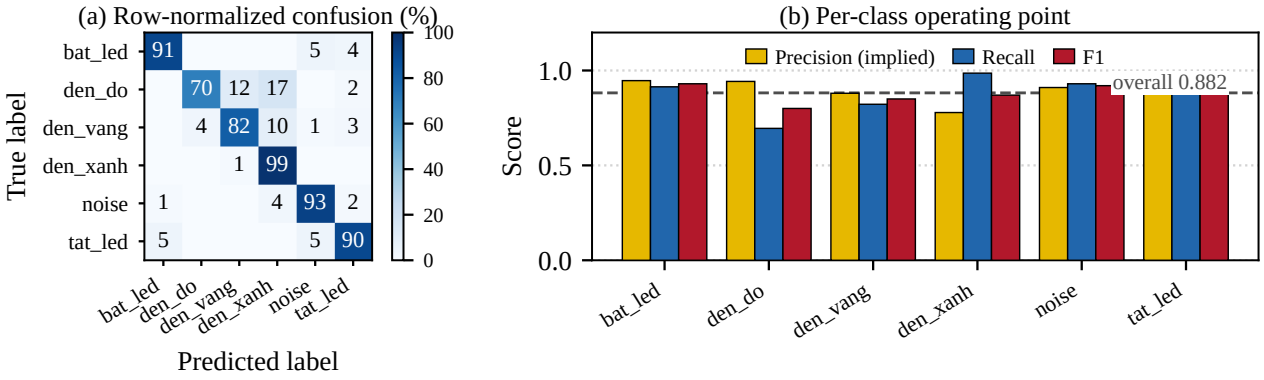


Fig. 4. Class behavior of the deployed classifier: (a) row-normalized confusion matrix and (b) per-class operating point.

TABLE VI
COMPARISON WITH PRIOR KEYWORD SPOTTING WORK ON MICROCONTROLLERS

System	Params	Acc. (%)	Latency	Task
González [5], ESP32	21.9 k†	87.14	28 ms	custom, Spanish
DS-CNN-S [2], M7	38.6 k	94.4	—	Speech Commands, 12 cl.
MLPerf Tiny [4], M4	24.9 k	92.2	—	Speech Commands, 12 cl.
MicroNet-KWS-S [3], M4	—	95.3	—	Speech Commands, 12 cl.
This work, ESP32	**1.88 k**	**88.2**	**7 ms**	custom, Vietnamese, 6 cl.

Parameter counts are derived from (4).

VII. DISCUSSION

The cost model determines the scaling. An added class widens only the dense layer, at 192 operations, negligible against the 3.23×10^5 operations of the front end, so vocabulary is limited not by arithmetic but by three factors. First, errors concentrate among phonetically similar commands, so accuracy falls with lexical density rather than class count. Second, a larger vocabulary requires more channels, and widening the first convolution from 8 to 32 channels quadruples its cost. Third, the confidence threshold is harder to satisfy as the softmax spreads over more classes. The design suits a closed set of ten to twenty phonetically distant commands, and tonal minimal pairs would require a pitch feature that the cepstral representation discards.

Four bounds qualify the results. The 88.2% is within-speaker recall on twelve minutes of clean audio from four speakers split over recordings, an upper bound on field accuracy. No result is reported under additive noise [13], at varying distances, or for an unenrolled speaker, and with a single female speaker, a gender-disjoint test is a one-sample estimate. No result is reported on a standard benchmark, so Table VI places the system in the design space rather than ranking it. The measurements cannot separate the code generator from the topology (Section V). Finally, Table V simulates the control logic. By construction, the policy loses no events and never diverges, although confirming this on hardware remains future work.

VIII. CONCLUSION

This paper reported a contactless lighting system performing keyword spotting, optical gesture recognition, and full-duplex web synchronization locally on a single ESP32. The system uses a two-layer integer-quantized network of 1878 parameters that attains 88.2% accuracy on a six-class Vietnamese command set and executes in $7\,\mathrm{ms}$ within a $3.8\,\mathrm{kB}$ arena and $31.2\,\mathrm{kB}$ of flash. The analytical contribution is a cost model that attributes these measurements. The front end incurs $12.6\times$ the network arithmetic, 70% of the arena and 75% of the flash are not model-attributable, and at most 6% of the inference interval is arithmetic. By Amdahl's law, the fourfold inference speedup reduces the full cycle by at most 7.9% against a measured 3.4%, so the stage-level gain does not translate into a comparable reduction of the command cycle, and the measurements cannot separate the roles of topology and code generator. The system contribution is an arbitration policy. In simulation, the naive scheme diverges in half of all browser-involving trials and loses one gesture in three during a burst, while the proposed policy shows neither fault.

REFERENCES

[1] Y. Abadade *et al.*, "A comprehensive survey on TinyML," *IEEE Access*, vol. 11, pp. 96 892–96 922, 2023.
[2] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello Edge: Keyword spotting on microcontrollers," arXiv:1711.07128, 2017.
[3] C. Banbury *et al.*, "MicroNets: Architectures for deploying TinyML on commodity microcontrollers," in *Proc. MLSys*, 2021.
[4] C. Banbury *et al.*, "MLPerf Tiny benchmark," *Proc. NeurIPS Datasets and Benchmarks*, 2021.
[5] M. González, S. Gutiérrez, R. Espinosa, and H. Ponce, "Deploying real time speech recognition on ESP32 using TinyML and Edge Impulse," preprint, 2025.
[6] P. Warden, "Speech Commands: A dataset for limited vocabulary speech recognition," arXiv:1804.03209, 2018.
[7] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. MLSys*, 2021.
[8] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs," arXiv:1801.06601, 2018.
[9] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer arithmetic only inference," in *Proc. IEEE CVPR*, 2018, pp. 2704–2713.
[10] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357–366, 1980.
[11] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.
[12] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification algorithms," *Neural Comput.*, vol. 10, no. 7, pp. 1895–1923, 1998.
[13] J. Thiemann, N. Ito, and E. Vincent, "The diverse environments multichannel acoustic noise database (DEMAND)," in *Proc. Meetings on Acoustics*, 2013.
[14] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, 2011.
[15] *PAJ7620U2 Datasheet*, PixArt Imaging, 2022.
[16] *INMP441 Datasheet*, InvenSense, 2015.